

Highlights

A Lightweight Networking Strategy for Robotic Control Systems

Jake Robert Read , Nicholas Stearns Selby , Patrick Wahl, Douglas Kogut

- **Stateless Multipath Routing** that enables autonomous node packet processing
- **Enhanced Network Robustness and Efficiency** optimizing network flow and reliability

A Lightweight Networking Strategy for Robotic Control Systems

Jake Robert Read ¹

*The MIT Center for Bits and Atoms, Room E15-401, 20 Ames
Street, Cambridge, 02139, MA, USA*

Nicholas Stearns Selby ¹

*Renewvia Solar Ethiopia PLC, Bole sub-city, Woreda 03, House No. 174/175, Saya
Building, 5th Floor, Ethiopia*

Patrick Wahl

210 Green Street, Cambridge, 02139, MA, USA

Douglas Kogut

77 Massachusetts Avenue, Cambridge, 02139, MA, USA

Abstract

Networked Control Systems (NCS) demand low-latency, deterministic, and fault-tolerant communication, yet existing solutions such as Switched Ethernet suffer from single points of failure and **provide no adaptive multipath routing without offline path planning and a centralized configuration entity**. We introduce TinyNet, an open-source networking protocol for Networked Control Systems, implementable on system endpoints with minimal complexity. Operating without global state, TinyNet achieves memory efficiency and fault tolerance. It facilitates real-time multipath routing via a busyness-metric cost function that weights hop count against downstream buffer occupancy, reducing routing bottlenecks and distributing load across available paths. TinyNet targets small packet sizes to reduce transmission overhead, making it well-suited for control-loop traffic dominated by single-packet messages. **Simulation of a 16-node mesh, calibrated to a 1.2% match against**

Email address: nicholas.selby@renewvia.com (Nicholas Stearns Selby )

¹Authors JR Read and NS Selby contributed equally to this work.

hardware measurements, shows TinyNet sustains about 1.9 times the load of single-path shortest-path routing that is not planned for the traffic, matching the capacity of routes planned in advance. After one to four simultaneous node failures, its median blackout is 1.2–3.9 ms with at most 13 packets lost per flow. With two or more failures, loop-free-alternate fast reroute with the fastest documented failure detection typically loses hundreds of packets while awaiting routing reconvergence. These results make TinyNet a candidate for distributed robotic control systems.

Keywords: Networked control systems, Control of networks, networked robotics, computer networks, fault-tolerant

1. Introduction

Networked control systems (NCS) are integral components of sophisticated robotics and avionics, orchestrating the collaborative function of numerous sensors and actuators to achieve objectives such as locomotion, stabilization, sensor fusion, and position control [1, 2]. Additionally, NCS play a pivotal role in manufacturing settings, interlinking multiple machines for synchronized material handling and production planning [3]. This field distinguishes itself from conventional networking paradigms in several key aspects:

1. **Prioritization of Message Efficiency Over Throughput:** Unlike traditional networks where throughput is paramount, NCS prioritize efficient message sizes, typically ranging from three to fifty bytes [1]. The emphasis is on the rapidity of message delivery, with many communications comprising single-packet messages.
2. **Critical Need for Deterministic Message Delivery:** In NCS, the stability of control loops is contingent upon the guarantee of message delivery within specific time frames [2]. This necessitates a network where the round trip times (RTTs) have minimal variability, ensuring reliable and predictable communication.
3. **Essential Robustness and Simplified Management:** Robustness in NCS is non-negotiable, requiring the absence of single points of failure [3]. Furthermore, these systems are designed to operate independently of network controllers for coordinating port-forwarding configurations. Avoiding central management for route adjustments on all

nodes is crucial, as it significantly reduces complexities and variations in RTTs.

In summary, NCS demand a unique networking approach focused on small, swift message exchanges, determinism in delivery times, and a robust, decentralized architecture to maintain system stability and efficiency.

Recent developments in the field of NCS have seen a gradual shift from traditional fieldbuses to more contemporary networking solutions. Currently, the state-of-the-art in NCS predominantly involves the use of Switched Ethernet or its derivatives in proprietary protocols [1, 2]. This transition has been driven by the greater accessibility of Switched Ethernet hardware and its status as an open standard. Such features facilitate the integration of control products from various vendors, enabling customers to build more scalable and complex systems without the need for extensive reconfiguration or adherence to FieldBus specifications.

Despite its widespread adoption, Switched Ethernet falls short in addressing the specialized requirements of NCS. Its design, primarily tailored for general-purpose networking, does not align perfectly with the unique needs of NCS, particularly in areas such as real-time data transmission and network robustness. In Section 2, we discuss a key limitation of Switched Ethernet: its inability to support more than one route to any given endpoint in a network graph. This constraint leads to bottlenecks at specific switches and introduces Single Points of Failure (SPoF) in network architectures. Consequently, a failure in a single link can render many endpoints irreversibly inaccessible.

To enhance Message Delivery Determinism, a key performance indicator for NCS, we propose the implementation of a multipath routing strategy. This approach allows for the operation of NCS within highly interconnected graph structures, as opposed to limited spanning-tree configurations. It enables the rerouting of messages around congested nodes instead of accumulating them in extensive buffers.

However, current multipath routing methods, predominantly designed for datacenter environments, rely heavily on network controllers (NCs) to manage and update port-forwarding tables. These NCs hold comprehensive information about the entire network’s state. In scenarios like a link failure or a sudden surge in network traffic (for instance, due to a router becoming overloaded), these controllers must reconverge to a new suitable routing topology and disseminate updated routing instructions across the network.

This reconvergence takes up to several hundred milliseconds with vanilla link-state protocols [4, 5]. Modern Fast Reroute pre-computation [6, 7] reduces this to tens of milliseconds, but only for failures the pre-computed alternates cover, and at the cost of maintaining per-destination backup state. TSN’s Frame Replication and Elimination for Reliability [8] achieves path redundancy without post-failure reconvergence by replicating frames across pre-configured disjoint paths, but it requires a Centralized Network Configuration entity to define those paths and does not adapt to congestion at runtime. Such delays, albeit brief, can disrupt the entire network operation temporarily, leading to potential failures in the control loops of an NCS. Therefore, these existing multipath routing methods, given their reliance on network controller-based reconvergence, are not ideally suited for application in NCS.

Taken together, the limitations identified above—single points of failure, spanning-tree bottlenecks, and controller-dependent reconvergence delays—motivate the design of a stateless, multipath routing protocol purpose-built for NCS. To the authors’ knowledge, TinyNet is the first protocol to provide multipath routing for NCS that adapts to both failures and congestion at runtime, without offline path planning or a centralized configuration entity. It enables multipath capabilities without relying on a network controller for managing forwarding tables, achieved through a port-forwarding algorithm that utilizes each router’s *local awareness* of neighboring nodes’ packet queues and historical data on packet transmissions observed at the router’s ports. We describe the protocol rules and evaluate performance through hardware experiments and simulation.

2. Related Work

TinyNet sits at the intersection of several mature research and engineering communities: deterministic industrial Ethernet, IP-layer fast reroute, congestion-aware multipath in datacenter fabrics, backpressure-based routing for multi-hop wireless networks, and mesh routing for constrained devices. We survey each in turn, identifying the specific gap that motivates a stateless, busyness-aware protocol for networked control.

2.1. Deterministic Industrial Ethernet

The dominant deployed alternatives to Switched Ethernet in NCS are the modified-Ethernet fieldbusses: EtherCAT, PROFINET IRT, SERCOS III,

POWERLINK, and CC-Link IE TSN, together with the avionics-grade AFDX family. [9, 10] EtherCAT, in particular, is widely used in industrial robotics and achieves sub-microsecond synchronization and cycle times below 100 μ s by processing frames on the fly: a single frame circulates around a logical ring, and each slave extracts and inserts its data as the frame passes through dedicated hardware. [11, 12] Reported jitter for ARM-based EtherCAT masters in robotic motion control is below 1 μ s [13]. AFDX, the deterministic Ethernet profile flying on the A350 and 787, provides bounded latency through statically allocated Virtual Links and dual redundant networks. [14]

These protocols achieve excellent determinism, but at a cost that TinyNet is explicitly designed to avoid. EtherCAT requires a master to own the cycle and constrains the topology to ring or line structures; AFDX statically provisions every Virtual Link offline, requires certified switch hardware, and uses redundancy rather than multipath (the two copies follow disjoint but fixed paths). Neither addresses the case of a richly meshed graph of peer endpoints discovering routes at runtime without a controller.

The IEEE 802.1 Time-Sensitive Networking (TSN) family of standards augments standard switched Ethernet with deterministic mechanisms and is now widely deployed in industrial automation, automotive, and aerospace settings. [15, 16] In particular, the 802.1Qcc enhancement defines fully centralized, fully distributed, and hybrid models for configuring TSN networks. [17]

Two TSN features are directly comparable to TinyNet’s claims and deserve closer attention than they have received in the NCS literature. First, FRER addresses fault tolerance by transmitting duplicated frames over a network, which may include disjoint paths, and eliminating duplicates at the receiver; this is a different point in the design space than TinyNet’s reactive flooding, but it accomplishes a similar goal without paying the cost of post-failure reconvergence. Second, TAS (802.1Qbv) achieves bounded jitter by computing a global gate schedule offline, an approach whose schedulability problem is NP-complete and which has spawned a large literature on SMT-based and heuristic synthesis. [18, 19, 20] TAS provides excellent determinism for traffic whose timing is known at design time, but reconfiguring the schedule in response to runtime conditions remains costly, and the schedule itself is typically computed by a Centralized Network Configuration entity. TinyNet targets the complementary regime: networks subject to unpredictable node failures where offline scheduling is impractical and where the cost of a centralized configuration entity is unacceptable.

The wireless counterparts to these wired standards, WirelessHART [21] and ISA100.11a [22], extend similar determinism guarantees to unlicensed radio links using time-slotted channel hopping. TinyNet does not target lossy wireless links but shares with these protocols the goal of operating without a centralized scheduler.

2.2. IP-Layer Fast Reroute

The 200 ms reconvergence figure cited for OSPF and SPB [23, 24] reflects the behavior of vanilla link-state protocols and was the relevant baseline at the time those measurements were taken. Modern IP networks largely avoid this latency through pre-computed local repair. Loop-Free Alternates (LFA [6]), Remote-LFA, and Topology-Independent LFA install a backup next-hop in the forwarding table at routing-protocol convergence time; when Bidirectional Forwarding Detection or a link-down signal triggers failover, the cutover is local and is **designed to complete in under 50 ms [7]. Because the switch to a pre-installed alternate is fast, total recovery time is dominated by failure detection. BFD detection time is the negotiated transmit interval multiplied by a detect multiplier [25]. The fastest vendor-documented configuration, hardware-offloaded BFD at $3.3 \text{ ms} \times 3$, detects failures in about 10 ms, and the vendor states convergence of under 50 ms [26]. Other platforms document minimum intervals of 50 ms or more with multipliers of at least 3 [27]. MPLS Fast Reroute provides comparable guarantees in MPLS-TE networks.**

These mechanisms narrow the gap between stateful and stateless recovery to a small factor for single failures, and the remaining gap comes from detection rate, not from the repair mechanism itself. What they do not remove is the structural cost of pre-computed repair: LFA computes alternates against a single failure assumption, coverage is topology-dependent (not every prefix admits a loop-free alternate, and extending coverage requires Remote LFA [28] or TI-LFA), and configuration complexity grows with SRLG (shared risk link group) constraints. For NCS, where control loops may run at kilohertz rates, each millisecond of blackout corresponds to one or more missed deadlines, so coverage gaps and multi-failure behavior matter as much as best-case cutover time.

The IETF Deterministic Networking (DetNet) architecture [29] provides a complementary IP-layer framework for bounded latency and packet loss, combining traffic engineering, resource reservation, and path redundancy. Unlike TinyNet, DetNet requires per-flow forwarding state at each node and

offline configuration of forwarding resources, placing it firmly in the stateful, controller-dependent category.

2.3. Congestion-Aware Multipath in Datacenters

The datacenter networking community has produced a rich family of congestion-aware multipath schemes that are relevant to TinyNet’s cost-function routing. ECMP [30] splits traffic across equal-cost paths but is oblivious to congestion. CONGA [31], implemented in custom ASICs, uses leaf-switch state plus piggybacked feedback from remote leaves to make flowlet-level path decisions and reacts to congestion on the order of microseconds. Hula [32] pushes this idea further by having each switch track only the best path to each destination through each neighbor, using probe packets in the data plane and running on programmable (P4) hardware. Other proposals like DRILL, LetFlow, Hedera, and CONGA-Flow trade off flow-completion time, asymmetry tolerance, and reordering sensitivity.

TinyNet’s cost function in equation (5) is structurally similar to CONGA’s path-utilization metric and to Hula’s per-neighbor congestion field, but the design constraints differ in three ways. First, the granularity is per-packet rather than per-flowlet, because NCS messages are typically a single packet and the notion of a flow does not usefully apply. Second, the state at each node is reduced to a single byte per neighbor (transmit buffer depth), eliminating the per-destination tables that Hula and CONGA maintain. Third, TinyNet runs on microcontroller-grade endpoints rather than programmable switches, which forces the protocol logic into roughly a hundred lines of C rather than a P4 program. These constraints rule out direct adoption of CONGA or Hula in their published form, but the analogy clarifies that TinyNet inherits and specializes a well-established line of work.

2.4. Backpressure Routing

The cost function used in TinyNet is, viewed from a different angle, a variant of backpressure routing. The original max-weight/differential-backlog algorithm of Tassiulas and Ephremides [33] routes packets in the direction of the steepest negative gradient of queue backlog and is throughput-optimal under broad stability conditions. Subsequent work by Neely et al. [34, 35] extended the framework to time-varying wireless channels and unified it with utility-optimal control. Practical realizations include BCP, the first deployed

backpressure protocol for wireless sensor networks [36], DiffQ for differential-backlog congestion control [37], and a body of follow-up work on link-feature variants [38].

TinyNet differs from canonical backpressure in two important ways. Pure backpressure requires a separate queue per destination at each node, which is impractical for a microcontroller serving dozens of endpoints; TinyNet collapses this to a single transmit buffer per link and a small LUT, trading off a guarantee of throughput optimality for memory efficiency. And TinyNet weights backlog against hop count via the tunable λ parameter rather than acting on backlog alone, which avoids the well-known last-packet delay problem of pure backpressure under light loads. We position TinyNet as a backpressure-inspired heuristic engineered for the NCS operating point rather than as a throughput-optimal scheme.

2.5. Mesh Routing for Constrained Devices

The IETF’s RPL [39] is the closest analog to TinyNet in the constrained-device space. RPL constructs a Destination-Oriented Directed Acyclic Graph (DODAG) rooted at a sink, with each node selecting a preferred parent according to a configurable Objective Function (typically Expected Transmission Count or hop count). Like TinyNet, RPL is designed for cheap nodes with limited memory and is robust to churn. Unlike TinyNet, RPL is optimized for low-power lossy links, assumes converge-cast traffic toward a root, and inherits some of the same single-tree pathologies as Spanning Tree once the DODAG is established. Reactive variants such as P2P-RPL [40] relax the converge-cast assumption but introduce on-demand route-discovery latency that NCS cannot tolerate. The mobile ad-hoc routing protocols AODV and DSR share the same trade-off: cheap to run, but with discovery latencies in the tens to hundreds of milliseconds on first contact.

TinyNet’s flood-on-LUT-miss behavior is reminiscent of AODV’s route-request flooding and of RPL’s DIO/DIS exchanges, but is specialized in two ways: floods carry payload (so the first packet to a new destination is not delayed by a separate discovery round), and floods are triggered by the absence of an LUT entry rather than by a periodic timer.

2.6. Position of TinyNet

Across these threads, TinyNet occupies a specific point:

- **Compared to industrial Ethernet** (EtherCAT, PROFINET IRT, AFDX, TSN): TinyNet sacrifices the sub-microsecond determinism and

certifiability of scheduled approaches in exchange for the ability to operate on arbitrary mesh topologies without offline configuration or a centralized configuration entity.

- **Compared to IP-FRR:** TinyNet’s single-failure blackout is about one third that of LFA with the fastest vendor-documented BFD configuration. The difference comes from TinyNet’s faster link-local detection. TinyNet also needs no per-destination state or pre-computed alternates, so it has no coverage gaps and tolerates multiple simultaneous failures.
- **Compared to CONGA/Hula:** TinyNet adopts the congestion-aware multipath philosophy but reduces per-node state to a single byte per neighbor and shifts the implementation target from programmable switch ASICs to commodity microcontrollers.
- **Compared to backpressure:** TinyNet is a memory-frugal heuristic in the same family, with a hop-count bias to prevent backlog-driven detours under light load.
- **Compared to RPL/AODV:** TinyNet trades the converge-cast assumption (RPL) and the separate discovery phase (AODV) for payload-carrying floods that amortize discovery into useful transmission.

This combination of properties—stateless, controller-free, multipath, busyness-aware, payload-carrying floods on discovery—does not appear together in any of the protocols we surveyed, and is what we evaluate in Section 5.

3. Theory

The TinyNet protocol significantly reduces the need for administrative communication between nodes. Rather than exchanging detailed topological data, nodes simply send periodic heartbeats to their immediate neighbors and route packets to designated ports, rendering the protocol stateless with respect to global topology. This approach allows TinyNet to operate effectively in both standard and failure situations, adopting a distributed method where the responsibility of message transmission is shifted to neighboring nodes instead of relying on pre-defined routes. We design TinyNet with the following constraints:

- **Endpoint Integration:** TinyNet is engineered for straightforward integration into robotic endpoints. Utilizing a UART link, a common peripheral in most microcontrollers, the protocol is encapsulated within a C library. This design ensures that it can be effortlessly incorporated into existing firmware setups.
- **Open-Source Development:** We have developed TinyNet as an open-source initiative, enabling wide applicability in the development of Networked Control Systems. By avoiding the creation of a proprietary standard, we envisage TinyNet being adopted in diverse hardware environments, thereby fostering various practical implementations.
- **Statelessness:** A core objective in TinyNet’s development is to minimize the need for centralized state awareness in routing processes. This principle not only bolsters the network’s overall resilience but also obviates the necessity for a central network controller or complex network configurations.
- **Real-Time Multipath Capability:** A significant focus of TinyNet is to provide real-time multipath routing capabilities. This feature is crucial for efficient port-forwarding and adaptive network traffic management.
- **Robustness:** A fundamental requirement of TinyNet is its consistent performance even in the face of link disruptions or router failures, ensuring reliability and continuity in network operations.

3.1. The TinyNet Algorithm

Each node handles incoming packets as follows:

```

if the packet is not a buffer update then
    update the LUT using packet src. and hop count
end if
if packet is Standard then
    if I am destination then
        process data in packet
        reply with ACK
    else
        increment hop count
        if LUT has destination address then
            send packet to port which minimizes C
        else

```

```

        change packet type to Standard Flood
        send packet to all ports
    end if
end if
else if packet is ACK then
    if I am destination then
        process acknowledgement
    else
        increment hop count
        if LUT has destination address then
            send packet to port which minimizes C
        else
            change packet type to ACK Flood
            send packet to all ports
        end if
    end if
end if
else if packet is Standard Flood then
    remove packet dest. from LUT at that port if it exists
    if I have not yet seen this flood then
        if I am destination then
            process data in packet
            reply with ACK
        else
            increment hop count
            if LUT has destination address then
                change packet type to Standard
                send packet to port which minimizes C
            else
                send packet to all other ports
            end if
        end if
    end if
end if
else if packet is ACK Flood then
    remove packet dest. from LUT at that port if it exists
    if I have not yet seen this flood then
        if I am destination then
            process acknowledgement
        else
            increment hop count
            if LUT has destination address then
                change packet type to ACK
                send packet to port which minimizes C
            else
                send packet to all other ports
            end if
        end if
    end if
end if

```

```

    end if
  end if
else
  write buffer depth to LUT
end if

```

where LUT is look-up table, ACK is acknowledgement, and C is the cost function defined in equation (5). Figure 1 illustrates the three core behaviors of TinyNet: heartbeat-based buffer-depth propagation, cost-function multi-path routing, and flooding-based failure recovery.

The packet handling algorithm identifies whether the incoming packet is a standard message or an ACK, whether or not it is a flood, and whether or not it is a heartbeat, all by reading the first byte of the packet. Packets are structured as shown in Table 1.

Table 1: TinyNet Packet Structure.

Type	8 b	10 b	6 b	10 b	6 b	N B
STD	0xFF or 0xFE	Dest.	HC	Src.	N	Msg.
ACK	0xFD or 0xFC	Dest.	HC	Src.	-	-
HB	Buffer Depth	-	-	-	-	-

Notes. STD = standard packet; ACK = acknowledgement; HB = heartbeat (single-byte; all remaining fields absent). Dest. = destination ID; HC = hop count; Src. = source ID; N = payload length in bytes; Msg. = message payload. Lowercase ‘b’ = bits; uppercase ‘B’ = bytes.

The first byte designates the message type: 0xFF = Standard, 0xFE = Standard Flood, 0xFD = ACK, 0xFC = ACK Flood. HB packets carry only a buffer depth in $[0, 251]$ to avoid ambiguity with the four other packet types.

3.2. Message Delay Determinism

A critical performance measure for TinyNet is its capability in Message Delay Determinism, referring to the speed and reliability with which a packet reaches its intended destination. The total time a packet spends at a node is quantifiable as:

$$D = D_{\text{rx}} + D_{\text{process}} + D_{\text{tx}} \quad (1)$$

where D_{rx} and D_{tx} denote the time taken to receive and transmit the packet, respectively, via UART, while D_{process} represents the processing time required for the packet. D_{process} is generally constant for all packet types except heartbeats and is measurable using a logic analyzer.

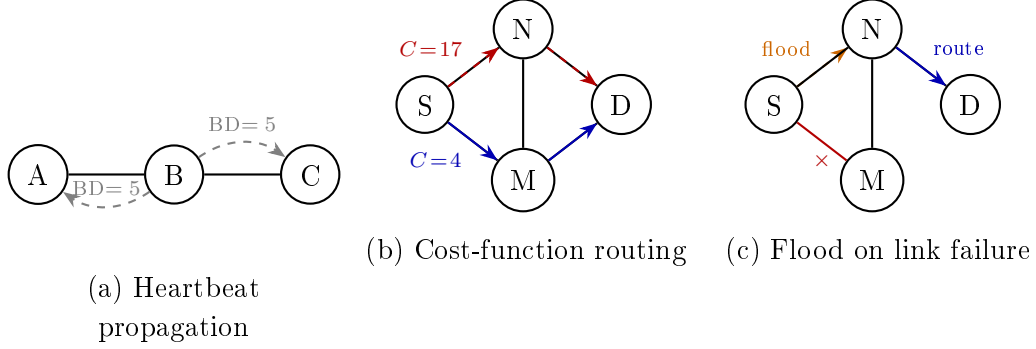


Figure 1: TinyNet behaviors. **(a)** Each node periodically broadcasts its transmit buffer depth (BD) as a single-byte heartbeat to immediate neighbors only. **(b)** A node routes each packet to the port minimizing $C = HC + \lambda \cdot BD$ (equation 5); here the path via M ($C = 4$) is selected over the congested path via N ($C = 17$). **(c)** When the link S–M fails, heartbeats from M cease; S floods the next packet to all remaining ports. Node N, which has a valid LUT entry for destination D, converts the flood to a standard packet and forwards it normally.

The reception time, D_{rx} , can be calculated as:

$$D_{rx} = P_\ell \times D_{byte} + L_{tt} \quad (2)$$

where P_ℓ is the packet length in bytes, D_{byte} is the processing time per byte in the UART buffer, and L_{tt} is the link transmission time, defined by:

$$L_{tt} = P_\ell \times 10/L_{br} \quad (3)$$

where L_{br} represents the link’s bitrate. Notably, each byte transmitted over UART consists of eight message bits plus a start and a stop bit, making a total of ten bits. Typically, D_{rx} is influenced predominantly by the bitrate. Equation (2) assumes the interrupt service routine completes before the next byte arrives; this holds at the 3.125 Mbps operating point (300 MHz core, 8-bit UART) but may be violated at higher bitrates.

Similarly, the transmission time, D_{tx} , is calculated by:

$$D_{tx} = P_\ell \times (K \times D_{byte} + 10/L_{br}) \quad (4)$$

where K is the count of ports used for sending the packet. K does not amplify the bitrate term because transmissions through each port occur concurrently.

In scenarios where a node receives multiple packets faster than they can be processed, a queue forms, increasing both the queue depth and the latency

between receiving and transmitting packets. Thus, nodes must signal neighboring nodes, using backpressure, about more efficient routing alternatives.

TinyNet employs periodic heartbeats for this backpressure mechanism. At set intervals, each node transmits a single byte representing its current transmit buffer occupancy to each immediate neighbor. Specifically, the buffer depth reported in a heartbeat is the number of bytes currently enqueued in the transmit buffer of the link connecting the sending node to the neighbor receiving that heartbeat. This information is not propagated further but is used locally to enhance the path cost function, integrating both the hop count (HC) and a “busyness metric” (BD):

$$C(HC, BD) = HC + \lambda \cdot BD \quad (5)$$

where the hop count (HC) is derived from earlier packets received from the destination node on this port, while buffer depth (BD) information comes from the heartbeat. The parameter $\lambda \geq 0$ (in units of hops per byte of buffer depth) is adjustable by the network designer and balances hop count against node congestion in routing decisions.

ACK packets are routed back to the original sender using the same cost-function routing as standard packets. When a node receives any packet (including the original request), it records the source address and the port on which the packet arrived in its LUT. This allows intermediate nodes to route ACK packets toward the original sender without flooding: each intermediate node simply looks up the sender’s address in its LUT and forwards the ACK along the port minimizing C .

3.3. Robustness to Failure

In traditional network protocols, the handling of node failures differs markedly between stateful and stateless systems. Stateful protocols, which require nodes to have comprehensive and up-to-date knowledge of the network topology, often face delays in disseminating information about node failures. For example, in a network utilizing OSPF in which each node must possess real-time topological information for routing, when a node failure occurs, the information about this failure must be communicated and processed by all other nodes, leading to potential delays.

Conversely, stateless network protocols typically employ a spanning tree to organize the network, which inadvertently introduces its own set of challenges. Should a node within this spanning tree fail, the sections of the

network downstream from that node become unreachable until the network detects the failure and reconstructs the entire tree.

TinyNet aims to merge the benefits of stateful network’s multipath routing with a more efficient recovery process in the event of failures. It utilizes heartbeat signals to foster a distributed awareness of the network’s state rather than relying on a centralized approach. Each node, at intervals longer than the expected heartbeat frequency, checks for recent heartbeats on each port. The reception of a heartbeat indicates a functioning connected node, while its absence suggests a potential failure, prompting the node to update its lookup table accordingly. This method allows each node to assess the state of its immediate connections locally.

However, heartbeats alone are insufficient for informing all nodes affected by a failed node. To address this, TinyNet includes a specific bit in each non-heartbeat packet, indicating whether it was part of a flood from the transmitting node. Nodes initiate packet flooding when their lookup table lacks a route to the destination, signaling that a previously viable path is no longer functional. When a node receives such a flooded packet, it understands that the corresponding port does not lead to the intended destination.

LUT entries are invalidated by two complementary triggers. First, when a flood packet for destination D arrives on port P , the node marks that path through P toward D as no longer reliable. Second, each node maintains a per-port liveness flag that is set whenever any heartbeat arrives. At each liveness-check interval (2 ms), a node inspects every port: if the flag is set it is cleared; if it is not set, all LUT entries for that port are invalidated, removing it from all routing decisions until new packets re-establish its routes.

In response to node failures, neighboring nodes quickly recognize the lack of heartbeats and eliminate any associated routes from their lookup tables. If a node then receives a packet destined for a route now known to be defunct, it floods this packet back, marking it accordingly. This process continues until the packet reaches a node with an alternative path to the destination, at which point standard routing resumes. In this manner, TinyNet ensures that only the nodes requiring this failure information—and precisely when they need it—are informed, optimizing the network’s response to node failures. To bound flood overhead, each packet carries a hop count that is incremented at every forwarding step; packets whose hop count exceeds `MAX_HOPCOUNT` (set to 6 in the current firmware) are discarded. A single flooded packet therefore generates at most $\sum_{h=0}^{\text{MAX_HOPCOUNT}} (\Delta - 1)^h$ transmissions in the

network, where Δ is the maximum node degree, with each branch terminating earlier if it reaches a node that already has a route to the destination. Because TinyNet routes each packet independently, packets in the same logical stream may arrive out of order if they traverse paths of different length; for the single-packet control messages that dominate NCS traffic, this is benign, but multi-packet messages require application-layer sequencing. Loop prevention relies on the same two mechanisms as LUT aging: flood-triggered invalidation removes stale entries that would sustain a routing loop within one round-trip, and the hop-count drop discards any packet that has circulated for more than `MAX_HOPCOUNT` hops regardless.

4. Implementation

4.1. Hardware

For the hardware implementation of TinyNet, we developed compact routers powered by the ATSAM570 microcontroller from Atmel/Microchip, which operates at a core clock speed of 300 MHz. The router schematic and board file are illustrated in Figs. A.1 and A.2 in the appendix. While this microcontroller is capable of supporting UART transmission speeds up to 18.75 Mbps, we observed that a reduction in bitrate to 3.125 Mbps was necessary to achieve stable and reliable data transmission. At higher bitrates, the inter-byte arrival interval becomes shorter than the UART interrupt service routine latency on the ATSAM570, causing receive buffer overruns; the 3.125 Mbps setting provides sufficient margin for the 300 MHz core to service each incoming byte interrupt before the next byte arrives.

To enhance the system’s resilience to electromagnetic interference (EMI), we incorporated the ISL3177 differential driver into the UART lines. Although this measure was not strictly required for our initial testing environment, it is a prudent preparatory step toward operating TinyNet in more challenging EMI conditions, such as those found near Brushless Motor Controllers. This consideration forms a part of our agenda for future development and testing, ensuring that TinyNet remains robust and reliable in a wide range of operational settings.

4.2. Simulation

We implemented TinyNet’s software simulation in JavaScript, running on a commodity Windows laptop. The underlying framework is adapted from Simbit, a discrete-event P2P network simulator originally designed

for blockchain research [41]; we replaced its packet format and propagation model with the UART timing equations (equations (1)–(4)) and defined topology as an array of adjacency lists. Each element represents the port connections of one node, allowing us to model arbitrary topologies (evaluated here on up to 16 nodes).

In the simulation, each node is represented as a “manager.” These managers handle incoming packets on each port, maintain their own lookup and buffer depth tables, and apply the forwarding algorithm to each incoming packet. Events are scheduled by a discrete-event simulation engine that advances the virtual clock between scheduled callbacks. We can also inject custom traffic patterns to explore TinyNet’s behavior under specific conditions.

5. Evaluation

To assess the effectiveness of TinyNet, we employ our hardware testbed, focusing on accurately timing key message passing intervals and associated delays. These empirical measurements serve to calibrate and inform our simulation model. The simulation, in turn, facilitates rapid iteration across a variety of network traffic scenarios. It enables us to meticulously analyze and quantify the routing paths of messages within the network, offering a detailed insight into TinyNet’s performance under different conditions. This combined approach of practical testing and simulation provides a comprehensive evaluation of TinyNet’s capabilities.

5.1. Hardware

For our hardware evaluation, we utilized the Saleae Logic 8 Logic Analyzer to capture logic level voltages from the microcontrollers. This was crucial for monitoring the router firmware at both the application and networking layers, where pin states (high or low) are indicative of significant event occurrences.

5.1.1. Processing Delay

Processing Delay, denoted D_{process} , is defined as the duration required for a router to determine the appropriate port for forwarding a received packet. To characterize this delay, we recorded signals at the link layer using the logic analyzer, tracking a packet’s journey through a single router. Specifically, D_{process} was measured as the interval between the end of an incoming packet

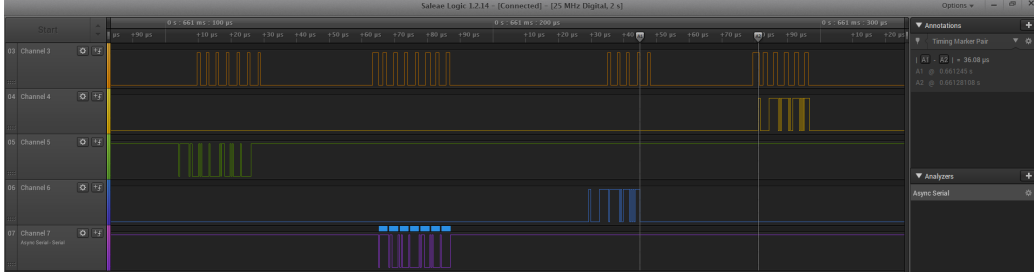


Figure 2: Logic-analyzer capture showing D_{process} measurement. Channels show UART activity: Channel 1 = incoming packet end, Channel 2 = outgoing packet start; the interval between them is $D_{\text{process}} = 37 \mu\text{s}$. Channel 3 shows per-byte interrupt handling ($D_{\text{byte}} = 1.5 \mu\text{s}$).

and the start of the outgoing packet’s transmission. By averaging results from 10 such packet traversals, we determined an average processing delay of $37 \mu\text{s}$. The small sample size is justified by the deterministic nature of the MCU’s execution path: the forwarding logic executes the same instruction sequence for every packet and runs at the exclusive priority of the main loop, with no competing interrupts during processing. Accordingly, trial-to-trial variation is negligible and the 10-sample mean is representative.

5.1.2. Link Transmission Time

Link Transmission Time, defined in equation (3) as L_{tt} , is the time taken for a packet to travel across a link. This calculation was verified using a logic analyzer. The transmission and reception of bytes also consume processor time. As the UART link handles bit-shifting asynchronously on each port, each received character triggers an interrupt requiring processor attention. We measured this interrupt handling time to be $D_{\text{byte}} = 1.5 \mu\text{s}$ per byte, consistent across all tested packets. This value can also be derived analytically: at 300 MHz with the interrupt service routine requiring approximately 450 clock cycles for context save, buffer copy, and return, the expected handling time is $450/300 \text{ MHz} = 1.5 \mu\text{s}$, confirming the measured result. This process is depicted in Fig. 2, Channel 3. We discuss in Section 5.2 how we account for these delays to align our simulation results with our hardware findings.

5.1.3. Router Behavior

To assess multi-link routing performance, we assembled a network comprising 12 interconnected TinyNet routers. This setup is illustrated in Fig. 3,

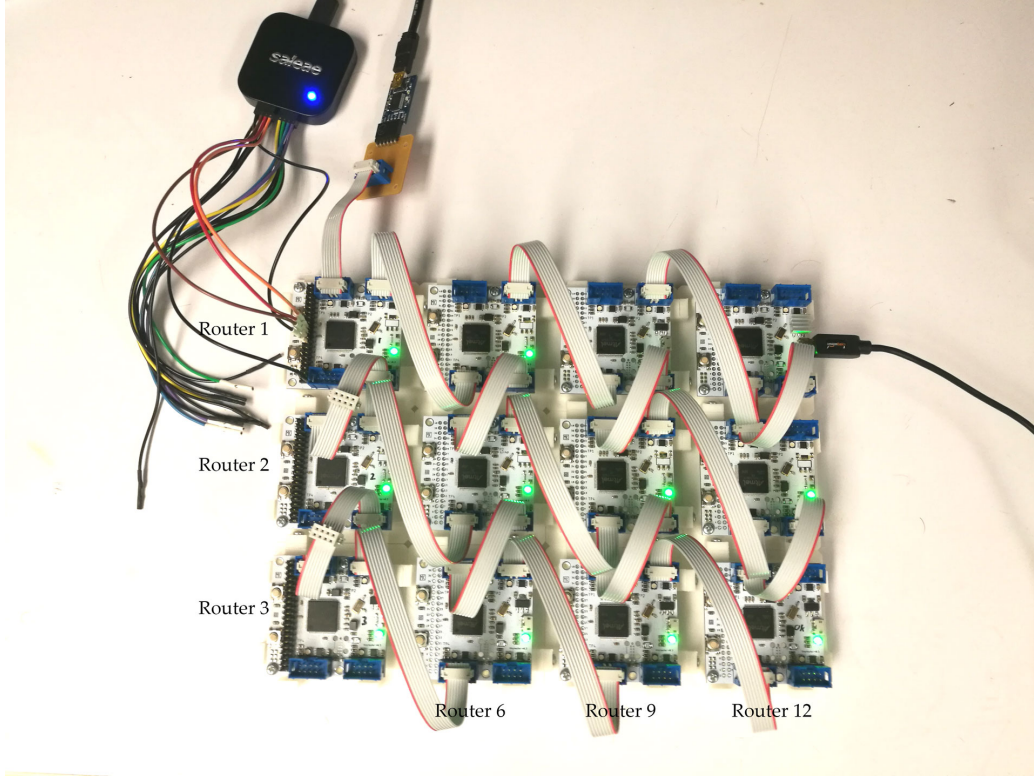


Figure 3: The 12-router hardware testbed. Each router is an ATSAM570-based TinyNet node. The Saleae Logic 8 logic analyzer (left) captures UART timing; the TinyNet Bridge (right, USB) provides host access. Routers are interconnected in a 3×4 grid topology.

along with the logic analyzer used for timing Round-Trip Time (RTT) and D_{packet} and verifying L_{tt} . Additionally, a TinyNet Bridge is shown, enabling network access via USB Port. We measured the RTT between Router 1 and Router 12, finding an average RTT of $511 \mu\text{s}$, captured directly by the logic analyzer and shown in Fig. 4. This hardware measurement is cross-validated by the simulation result of $505 \mu\text{s}$ (Section 5.2), a discrepancy of only 1.2%, which provides strong corroborating evidence for the accuracy of both measurements.

5.2. Simulation Validation

Our simulation, capable of representing various network topologies and traffic conditions, operates in JavaScript and leverages the open-source Sim-bit architecture [41]. To validate the simulation’s effectiveness, we emulate

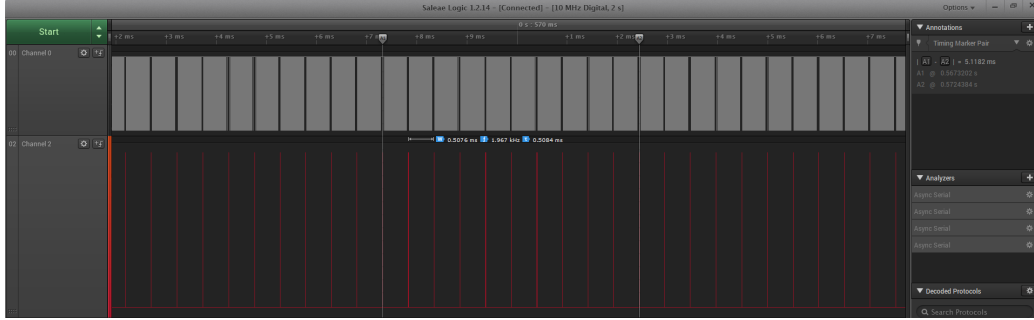


Figure 4: Logic-analyzer capture of the $511\mu s$ round-trip time between Router 1 and Router 12 in the 12-router hardware testbed. The cursors mark the start of the outgoing packet and the end of the returning acknowledgement; this interval is the primary hardware calibration point for the simulation model.

a 12-node grid network, mirroring our hardware setup. We initialize the simulated nodes with parameters derived from hardware measurements via the logic analyzer: processing delay $D_{\text{process}} = 37\mu s$, per-byte interrupt time $D_{\text{byte}} = 1.5\mu s$, and link bitrate $L_{\text{br}} = 20\text{ MHz}$. All simulations use $\lambda = 0.5$, a heartbeat transmission period of 1 ms, and a missed-heartbeat detection window of 2 ms. This process ensures that our simulation closely resembles real-world hardware operations.

We facilitate message transmission diagonally across the grid and record the round-trip time (RTT). Comparing the RTT of $511\mu s$ obtained from our hardware tests with the simulation’s RTT of $505\mu s$, we observe a minor discrepancy of only 1.2%, confirming that the simulation accurately reproduces hardware behavior at this operating point. RTT variance and failure-recovery statistics reported in subsequent sections are derived from simulation; hardware validation of these statistics remains future work.

5.3. Simulation Sensitivity Analysis

To bound the effect of hardware timing uncertainty on the reported variance results, we conducted a one-at-a-time sensitivity analysis, independently perturbing each of the three calibrated parameters— D_{process} , D_{byte} , and L_{br} —by $\pm 15\%$ from their hardware-measured values and re-running the 10-kHz cross-traffic scenario on the 4×4 grid. Each configuration uses the same steady-state method as the grid evaluation of Fig. 5.

All six perturbations remain stable at $\pm 15\%$, confirming that the simulation results are robust to plausible hardware timing uncertainty. Bitrate

Table 2: Sensitivity of per-hop RTT mean and standard deviation to $\pm 15\%$ perturbation of hardware timing parameters. Baseline: $D_{\text{process}} = 37 \mu\text{s}$, $D_{\text{byte}} = 1.5 \mu\text{s}$, $L_{\text{br}} = 20 \text{ MHz}$. Statistics are taken in steady state ($t \geq 50 \text{ ms}$ of a 200 ms simulation).

Configuration	Mean (μs)	σ (μs)
Baseline	104	25
$D_{\text{process}} - 15\%$	101	23
$D_{\text{process}} + 15\%$	170	43
$D_{\text{byte}} - 15\%$	125	30
$D_{\text{byte}} + 15\%$	144	31
$L_{\text{br}} - 15\%$	152	40
$L_{\text{br}} + 15\%$	104	25

and per-byte interrupt time have a moderate influence: a $\pm 15\%$ change in L_{br} changes σ by 0% to +60%, and a $\pm 15\%$ change in D_{byte} by +20% to +24%. The asymmetric response of L_{br} reflects TinyNet’s load-dependent path choices, which change as timing shifts. Processing delay has the strongest influence: a -15% perturbation leaves σ nearly unchanged (-8%), while a $+15\%$ perturbation increases σ by 72%.

D_{process} is validated by two independent methods: direct logic-analyzer measurement ($37 \mu\text{s}$) and analytical derivation from the MCU’s interrupt service routine ($450 \text{ cycles} \div 300 \text{ MHz per-byte}$, with the forwarding path executing a deterministic instruction sequence). The 1.2% RTT match at the hardware calibration point confirms this value, bounding the practical uncertainty in D_{process} to well under 15%. The parameters for which the analysis shows benign sensitivity, D_{byte} and L_{br} , are less tightly constrained by independent measurement but do not materially affect the reported results within their plausible uncertainty ranges.

5.4. Determinism of Communication Time

A crucial measure of TinyNet’s effectiveness is communication time determinism, specifically the standard deviation of the round trip time for each hop a message makes. Different from previous systems that relied on spanning tree models, TinyNet thrives in more extensive network topologies. Therefore, our evaluation focuses on both determinism and robustness within a compact grid setting.

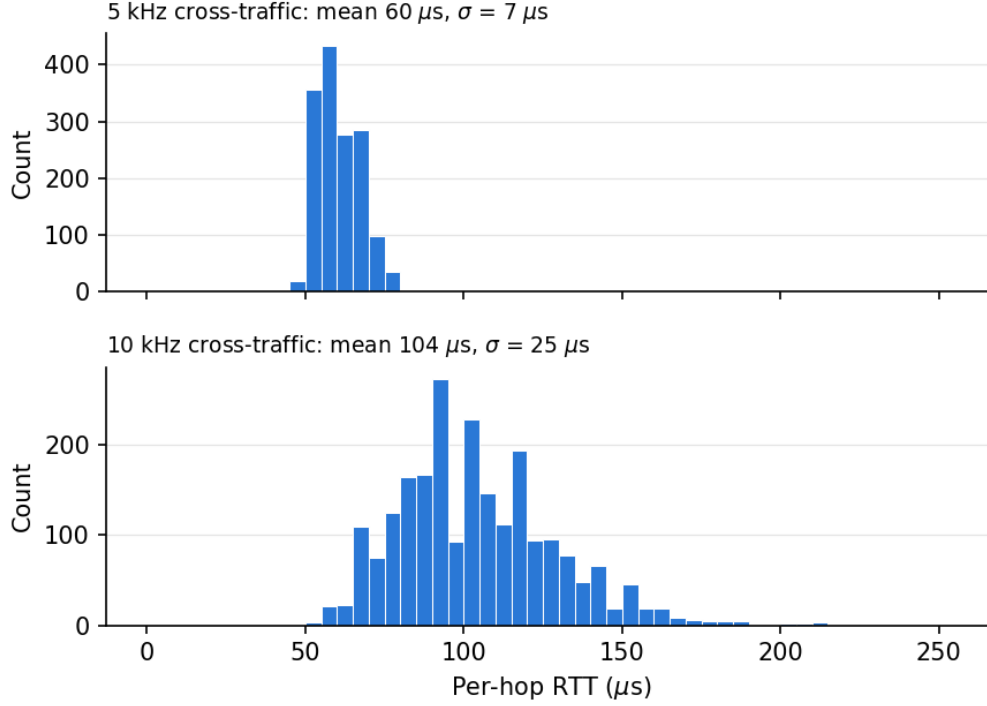


Figure 5: Per-hop RTT histograms in a 4×4 grid in steady state ($t \geq 50$ ms of a 200 ms simulation). Main flow: 5 kHz from node 0 to node 15. Cross-traffic on the opposite diagonal (node 3 to node 12) at 5 kHz (top) and 10 kHz (bottom).

5.4.1. Evaluation in a 4-by-4 Grid

We begin by simulating a 4-by-4 grid of nodes, with each node having a packet delay time of 37 μs (the hardware-measured D_{process}) and each link operating at a bitrate of 20 MHz. We transmit crucial messages diagonally across the grid at 5 kHz while cross-traffic runs on the opposite diagonal at 5 or 10 kHz in separate simulations. Each simulation runs for 200 ms after a warm-up exchange, and all statistics are taken from $t \geq 50$ ms so that they describe steady-state behavior rather than the start-up transient. The resulting per-hop RTT histograms are shown in Fig. 5. With 5-kHz cross-traffic, the mean per-hop RTT is 60 μs with $\sigma = 7 \mu\text{s}$; with 10-kHz cross-traffic, the mean is 104 μs with $\sigma = 25 \mu\text{s}$. If cross-traffic frequency increases beyond network capacity, no stable distribution exists.

5.4.2. Evaluation in a Simulated Airplane Wing Network

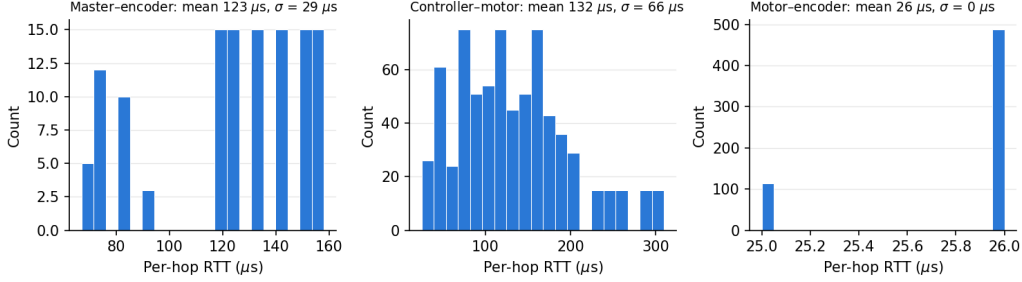


Figure 6: Per-hop RTT histograms in a simulated airplane wing topology (one master node, three controllers, eight motors, and eight encoders) in steady state ($t \geq 50$ ms of a 200 ms simulation), with traffic at 20% of the nominal rates: master–encoder queries (left), controller–motor loops (center), and motor–encoder loops (right).

In addition, we assess determinism within a simulated airplane wing network, structured into four layers: a single master node on top, three controllers in the second layer, each connected to the master and to all eight motors in the third layer, and each motor linked to its neighboring motors and an encoder in the fourth layer. The nominal traffic has each motor running a 500-Hz control loop with its encoder, each controller running a 200-Hz loop with every motor, and the master querying each encoder at 100 Hz. Using the steady-state method of the grid evaluation, the per-hop RTTs in Fig. 6 have a standard deviation of 29 μ s for master–encoder queries, 66 μ s for controller–motor loops, and under 1 μ s for motor–encoder loops. Supporting the nominal rates would require either faster nodes or a topology that spreads controller traffic across more links.

These results highlight two key outcomes:

- **Impact of Network Traffic on Determinism:** An increase in network traffic tends to reduce network determinism. As the number of packets traversing the network grows, so does the buffer depth at individual nodes, leading to increased message latency and variability in latency. This relationship is compounded by path length: networks with more routing hops expose each packet to more opportunities for queuing, so determinism degrades faster per unit of added traffic in larger networks. This is consistent with the airplane wing results, where the single-hop motor–encoder loops achieve $\sigma < 1 \mu$ s while the multi-hop

master–encoder and controller–motor paths, which cross the heavily loaded controllers, exhibit $\sigma = 29$ and $66 \mu\text{s}$.

- **TinyNet’s Adaptability:** Up to its capacity, TinyNet keeps per-hop RTT bounded under cross-traffic by distributing load across paths.

5.5. Stateful Routing Baseline

To directly address the question of how TinyNet compares against stateful routing, we implemented a shortest-path baseline in a JavaScript simulator. The baseline pre-computes BFS shortest paths at startup and routes each packet along the single lowest-hop-count path, with no multipath. Where several shortest paths exist, BFS takes the first in port order, which does not consider the traffic. We therefore also evaluate the baseline with *planned routes*: for the traffic pattern of Section 5.6, we searched every combination of shortest paths for the four flow legs (two data flows and their acknowledgements) and selected the one that minimizes the peak processing load on any node. Planned routes represent a stateful network engineered in advance for known traffic. All comparisons use the same topology, traffic, timing parameters, and failure schedules for both protocols.

Failure handling in the baseline models Loop-Free Alternate (LFA) fast reroute [6, 7] with Bidirectional Forwarding Detection (BFD) [25]. BFD detection time is the negotiated transmit interval multiplied by the detect multiplier, and vendor-documented lower limits on both vary widely. The most aggressive configuration we found documented is hardware-offloaded BFD at a 3.3-ms interval with a multiplier of 3 (≈ 10 ms detection), for which the vendor states a total convergence of under 50 ms [26]. More common limits are far longer: a 50-ms minimum interval with a minimum multiplier of 3 [27], or recommended minimums of 100–300 ms with a multiplier of 3 [27]. We use two failure models. The *cutover-window* model (Section ??) drops all packets on the failed path for a window T_{LFA} and then resumes on recomputed shortest paths, as if an alternate always existed. The *coverage-aware* model (Section ??) is more realistic: until BFD detects the failure 10 ms after it occurs, neighbors keep forwarding into the dead links; at detection, each neighbor of a failed node switches affected destinations to an RFC 5286 alternate computed on the pre-failure topology (node-protecting where one exists, otherwise link-protecting), or drops them if no alternate exists; and all nodes recompute routes on the post-failure topology after

an IGP reconvergence time of 200 ms, the OSPF reconvergence figure cited in Section 2.2. A 64-hop time-to-live discards packets caught in transient loops. We compare against LFA rather than RPL because LFA represents the fastest-recovering class of stateful protocols in production use; RPL’s DODAG repair mechanism targets energy-constrained wireless links and operates on a fundamentally different latency regime not directly applicable to UART-connected NCS (see Section 2.5).

5.6. Traffic Capacity

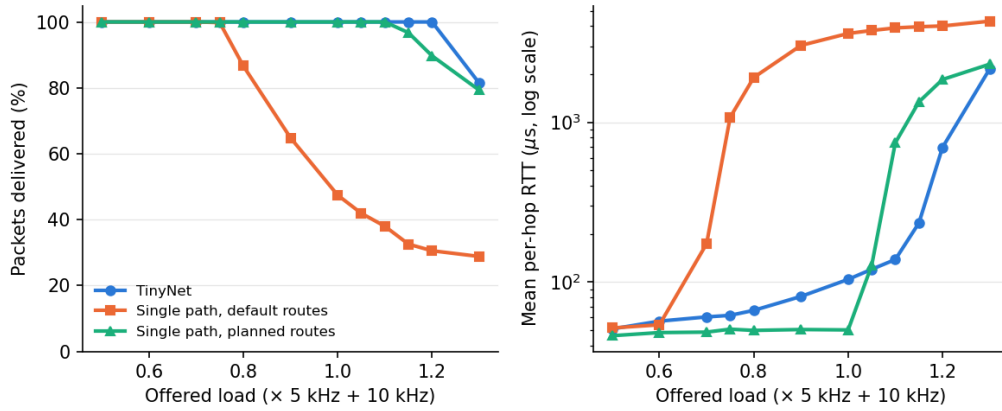


Figure 7: Delivery ratio (left) and mean per-hop RTT (right, log scale) on the 4×4 grid versus offered load, as a multiple of a 5 kHz main flow (node 0 to node 15) plus 10 kHz cross-traffic (node 3 to node 12): TinyNet (blue circles), single-path routing with the default BFS tie-break (orange squares), and single-path routing with routes planned for this traffic (green triangles). Each point is a 200 ms simulation; statistics use $t \geq 50$ ms.

A comparison of RTT variability between protocols is meaningful only at loads where both are stable. We therefore compare capacity: the highest offered load at which a configuration delivers at least 99% of packets and its mean RTT drifts by less than 20% between the first and last 50 ms of the analysis window. Fig. 7 shows delivery ratio and mean per-hop RTT as the load is scaled from 0.5 to 1.3 times the 5 kHz plus 10 kHz traffic of Fig. 5.

Single-path routing with the default tie-break is stable only up to 0.6 times this load. BFS sends both flows and their acknowledgements through the same column of nodes, overloading them; at 1.0 times the load, the baseline delivers 47% of packets with a mean per-hop RTT of 3.6 ms. By an analytic estimate of per-node processing load, 79% of all combinations of

shortest paths for this traffic overload at least one node, so this outcome is typical of single-path routing that does not account for the traffic. TinyNet is stable up to 1.15 times the load, about 1.9 times the capacity of the default single-path baseline, without any knowledge of the traffic. Single-path routing with planned routes is stable up to 1.0 times the load, comparable to TinyNet; we do not claim a difference between the two, because in both cases the busiest node is a flow endpoint, which no routing choice can relieve, and the remaining gap is within what small differences between the two simulator implementations could produce.

Below saturation, planned single-path routes deliver lower and more consistent per-hop RTT than TinyNet (e.g., $50 \pm 0 \mu\text{s}$ versus $104 \pm 25 \mu\text{s}$ at 1.0 times the load), because periodic traffic on non-conflicting paths never queues, while TinyNet’s cost function spreads packets over paths of varying length and occupancy. TinyNet’s advantage is therefore not lower jitter than a network engineered for known traffic. It is that TinyNet reaches the capacity of such a network without planning, whereas single-path routing that is not planned for the traffic usually collapses well below it.

5.7. Robustness to Single Node Failure

We first isolate the effect of failure detection with the cutover-window model, a single failure of node 7 (on the baseline’s shortest path), and a 5-kHz corner-to-corner flow with no cross-traffic. We evaluate four values of T_{LFA} : (i) 10 ms, the detection time alone of the fastest documented BFD configuration, which ignores switchover time and is thus a best case for LFA; (ii) 50 ms, the vendor-stated convergence bound for that configuration; (iii) 150 ms, the detection time of the 50 ms $\times$ 3 minimum configuration; and (iv) a hypothetical 3 ms, below any documented BFD configuration, chosen to match the detection budget of TinyNet’s own heartbeat mechanism (1-ms heartbeat period, 2-ms liveness check). In this scenario RFC 5286 provides a node-protecting alternate on both the forward and return paths, so the model’s assumption that an alternate exists holds.

Because TinyNet detects failures with a periodic liveness check, its blackout depends on when the failure occurs relative to that check. We therefore repeated the TinyNet run with the failure injected at 20 times spaced 0.1 ms apart across one 2 ms check period. Table 3 reports the blackout and the number of packets lost. TinyNet’s blackout averaged 3.7 ms (range 2.7–4.7 ms), with an average of 14 packets lost (range 10–19). Against the best-case documented LFA configuration ($T_{\text{LFA}} = 10 \text{ ms}$), TinyNet’s blackout

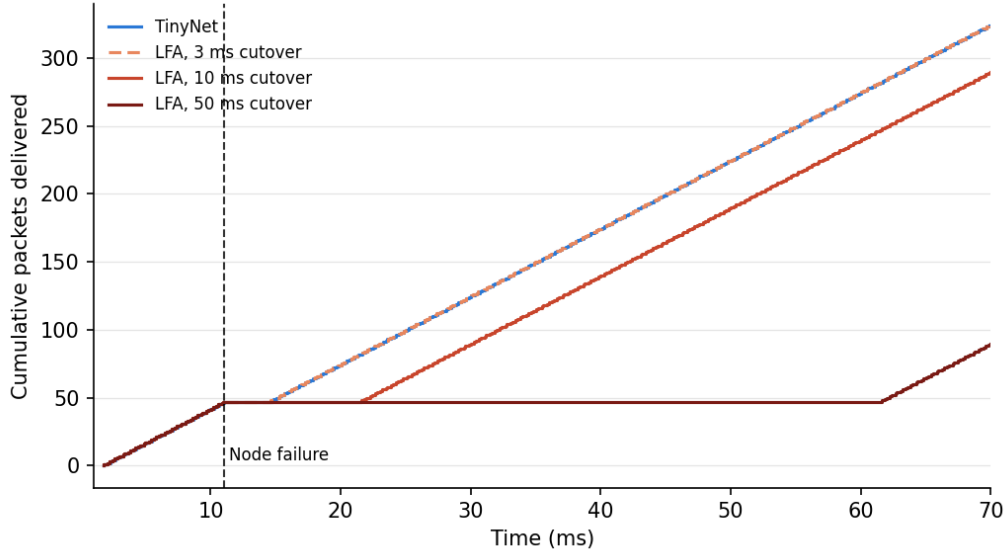


Figure 8: Cumulative packets delivered (y -axis; x -axis in ms) for a 5 kHz corner-to-corner flow following a single interior node failure at $t = 11.0$ ms: TinyNet (blue) versus the cutover-window LFA baseline with $T_{\text{LFA}} = 3, 10$, and 50 ms (red, light to dark). All lines overlap before the failure. The failure time was chosen so that TinyNet’s blackout (3.7 ms) is closest to its mean over 20 failure times. TinyNet’s curve nearly coincides with that of the hypothetical 3 ms LFA. See Table 3.

is about one third as long and it loses between a third and a quarter as many packets; against the vendor-stated convergence bound (50 ms), it loses about one eighteenth as many.

The hypothetical 3-ms configuration illustrates the origin of this advantage. When LFA is given the same detection budget as TinyNet, its blackout and packet loss match TinyNet’s average. For a single failure, then, TinyNet’s recovery time is set by its link-local detection latency, not by any inherent speed advantage of flooding over pre-computed alternates. TinyNet’s practical advantage is that its heartbeat runs on every link at a rate that BFD implementations do not document, and that it needs no per-destination state, no pre-computed alternates, and no alternate coverage.

5.8. Multiple Simultaneous Failures

To evaluate recovery from multiple failures without relying on an assumed alternate, we compared TinyNet with the coverage-aware LFA baseline in an

Table 3: Single-node-failure blackout (longest post-failure gap between deliveries) and packets lost for the 5 kHz corner-to-corner flow. A packet is lost if its acknowledgement never returns. TinyNet values are the mean (range) over 20 failure times spanning one liveness-check period. LFA blackouts do not depend on failure time in the cutover-window model and exceed T_{LFA} by the 0.4 ms in-flight time of the lost packets.

Protocol	Blackout (ms)	Packets lost
TinyNet	3.7 (2.7–4.7)	14 (10–19)
LFA, $T_{\text{LFA}} = 3$ ms (hypothetical)	3.4	14
LFA, $T_{\text{LFA}} = 10$ ms (3.3 ms $\times$ 3 BFD)	10.4	49
LFA, $T_{\text{LFA}} = 50$ ms (stated convergence)	50.4	249
LFA, $T_{\text{LFA}} = 150$ ms (50 ms $\times$ 3 BFD)	150.4	749

experiment in which LFA’s network design and analysis were fixed before the experiment was run. Both flows run at 3-kHz main flow and 6-kHz cross-traffic, the highest load at which every configuration is stable before any failure, so that failure recovery is not confounded with overload. For each of $k = 1, 2, 3, 4$ simultaneous failures, we ran 30 trials, each failing k nodes drawn uniformly without replacement from the 12 nodes that are not flow endpoints, at a time drawn uniformly across one 2-ms liveness-check period. Both protocols saw identical trials. A packet counts as lost if no acknowledgement for it returns by any path. A trial in which any flow’s endpoints are disconnected by the failures is excluded entirely for both protocols. A flow counts as recovered if at least 95% of the packets it sends in the last 50 ms of the run are delivered. Fig. 9 and Table 4 summarize the results.

With a single failure, LFA often loses nothing: when the failed node is off a flow’s path, or an RFC 5286 alternate covers it, delivery continues. In 18 of the 60 single-failure flows, however, no usable alternate existed, and the flow was dark until IGP reconvergence, losing up to 1,199 packets; TinyNet lost at most 19 packets in any single-failure flow. With two or more failures, the median LFA flow waits for reconvergence (about 600 packets lost for the 3-kHz flow and 1,200 for the 6-kHz flow), whereas TinyNet’s median loss stays between 9 and 13 packets with a median blackout of 3–4 ms. Comparing the same flow in the same trial, TinyNet lost fewer packets than LFA in 47%, 75%, 81%, and 85% of flows for $k = 1$ to 4, and more packets in 10%, 7%, 0%, and 12%. The LFA baseline’s losses scale with the assumed 200 ms reconvergence time; a faster-converging IGP would shrink, but not remove,

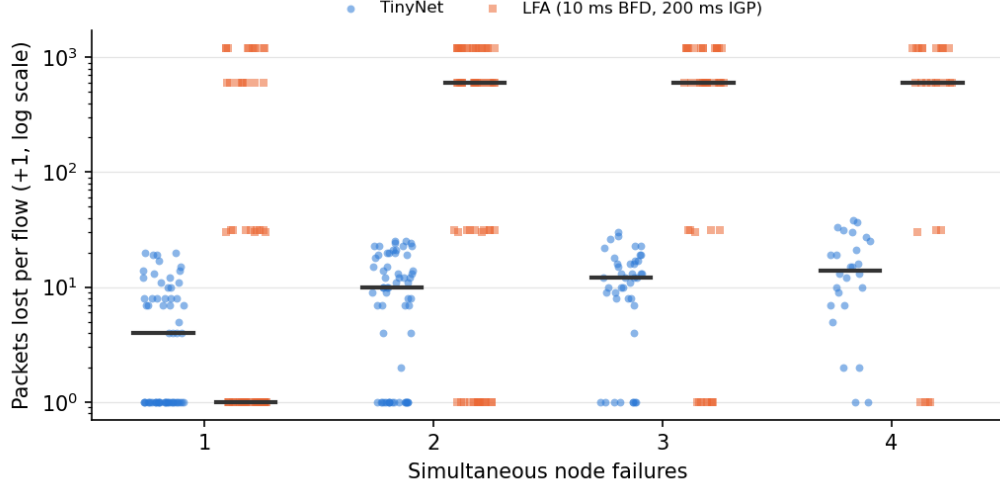


Figure 9: Packets lost per flow (plus one, log scale) after 1–4 simultaneous random node failures on the 4×4 grid with a 3-kHz main flow and 6-kHz cross-traffic: TinyNet (blue circles) versus the coverage-aware LFA baseline with 10 ms BFD detection and 200 ms IGP reconvergence (orange squares). Each point is one flow in one of 30 trials per failure count; bars mark medians.

this gap.

Key observations from these results include:

- **Rapid Recovery from Failures:** After one to four simultaneous node failures, TinyNet’s median blackout is 1.2–4.3 ms and its median loss is at most 13 packets per flow, with no knowledge of the topology and no pre-computed alternates.
- **Impact of Multiple Node Failures:** Loss and blackout grow only modestly with the number of failures, whereas an LFA network typically waits for full reconvergence once more than one node fails.
- **Capacity for Message Maintenance in Failures:** TinyNet sustains message transmission even under substantial network failures. In contrast to spanning-tree protocols, which lose connectivity to entire sub-trees on a single node failure [24], TinyNet reroutes around failed nodes by flooding.

Table 4: Packets lost and blackout per flow after k simultaneous node failures: median [interquartile range], and the number of flows that did not recover, for TinyNet and the coverage-aware LFA baseline (10 ms BFD detection, 200 ms IGP reconvergence).

k	Protocol	Packets lost	Blackout (ms)	Not recovered
1	TinyNet	3 [0–8]	1.2 [0.5–2.7]	0
	LFA	0 [0–599]	0.5 [0.3–200.3]	0
2	TinyNet	9 [0–17]	3.3 [0.6–4.3]	0
	LFA	598 [22–1197]	200.3 [7.8–200.5]	0
3	TinyNet	11 [7–16]	3.5 [2.9–4.5]	0
	LFA	600 [30–1198]	200.5 [10.6–200.6]	0
4	TinyNet	13 [6–23]	4.3 [3.1–5.7]	0
	LFA	599 [172–1197]	200.4 [58.1–200.6]	0

6. Conclusion

This paper introduces TinyNet, a novel routing protocol tailored for Networked Control Systems, emphasizing Message Delivery Time Determinism over Total Throughput. TinyNet’s efficacy in routing messages using minimalistic hardware has been showcased alongside a simulation tool that assesses its performance. A standout feature of TinyNet is its ability to manage message routing in extensively connected networks without relying on a Network Controller or requiring nodes to possess complete network topology knowledge.

The central contribution of this work is the creation of a port-forwarding strategy that utilizes real-time data about queue sizes at subsequent hops. This innovation allows for the intelligent rerouting of packets to circumvent congested network segments.

Determining realistic usage profiles for Networked Control Systems has been challenging due to limited existing research. We plan to gather primary data on practical applications and challenges in this field by consulting industry stakeholders at companies such as Boeing, Airbus, and Moog Inc.

The use of UART presents limitations in our project, specifically in capping the permissible bitrate to only 3.125 Mbps and necessitating a “configuration-dependent” environment where all ports must share the same baudrate settings. UART is an implementation choice driven by microcontroller availability; the routing protocol itself is link-layer agnostic, requiring only reliable byte-serial full-duplex delivery on each port. To address these issues, one future direction could include an FPGA-based link layer, implementing a

“co-clocking” technique for establishing bitrate. In this approach, neither side of a transmission pre-defines a bitrate. Instead, data transfer occurs in a responsive manner: once one side shifts data into a register, it sends an acknowledgment bit, triggering the transmission of the next bit. This method has successfully achieved bitrates of up to 65 Mbps. We have integrated this system with a microcontroller analogous to the ATSAM70 used in the TinyNet router, achieving similar transmission speeds via a 5-MHz wide parallel bus. This advancement aims to establish a completely stateless, configuration-free system, significantly enhancing the flexibility and scalability of TinyNet’s network infrastructure.

In addition to leveraging FPGAs for a stateless link layer, FPGAs can also be used for switching and port-forwarding, exploiting the simplicity of the TinyNet protocol for implementation in Verilog. This effort is expected to surpass the performance of Switched Ethernet while preserving the stateless, fault-tolerant, and adaptive qualities of TinyNet.

Our evaluation also identifies two limitations. First, TinyNet floods packets for which it has no route, so when failures partition the network, traffic addressed to an unreachable destination can congest surviving paths until the hop-count limit is reached; detecting when a destination has become permanently unreachable and suppressing further floods is an open design challenge. Second, for periodic traffic known in advance, single-path routes planned for that traffic deliver lower jitter than TinyNet at the same capacity, so TinyNet is best suited to networks whose traffic or failures cannot be planned for.

The evaluation in this paper is limited to networks of up to 20 nodes. Larger networks offer greater path diversity, which should allow TinyNet’s cost-function routing to distribute load more effectively; however, LUT memory grows as $O(N \cdot \Delta)$ per node and behavior under large-scale failures has not been measured, so evaluation at larger scales remains future work.

Finally, the current routing protocol employs the cost function defined in equation (5) for navigating around busy ports. We aim to explore machine learning techniques to refine this function, optimizing the routing process based on network topology and cost considerations, thereby enhancing overall network performance.

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

During the preparation of this work the authors used GPT4 in order to improve readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Funding Sources

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] R. A. Gupta, M.-Y. Chow, Networked control system: Overview and research trends, *IEEE Transactions on Industrial Electronics* 57 (7) (2009) 2527–2535.
- [2] J. R. Moyne, D. M. Tilbury, The emergence of industrial control networks for manufacturing control, diagnostics, and safety data, *Proceedings of the IEEE* 95 (1) (2007) 29–47.
- [3] S. Vitturi, C. Zunino, T. Sauter, Industrial communication systems and their future challenges: Next-generation ethernet, iiot, and 5g, *Proceedings of the IEEE* 107 (6) (2019) 944–961.
- [4] V. Eramo, M. Listanti, A. Cianfrani, Multi-path ospf performance of a software router in a link failure scenario, in: 2008 4th International Telecommunication Networking Workshop on QoS in Multiservice IP Networks, IEEE, 2008, pp. 197–203.
- [5] J. Farkas, Z. Arató, Performance analysis of shortest path bridging control protocols, in: *GLOBECOM 2009-2009 IEEE Global Telecommunications Conference*, IEEE, 2009, pp. 1–6.
- [6] A. Atlas, A. Zinin, Basic specification for IP fast reroute: Loop-free alternates, Tech. rep. (2008).
- [7] S. Kar, N. Kapoor, Understanding LFA and remote LFA IP fast reroute, <https://www.cisco.com/c/en/us/support/docs/multiprotocol-label-switching-mpls/ldp/212697-loop-free-alternate-and-remote-loop-free.html>.
- [8] C/LAN/MAN - LAN/MAN Standards Committee, IEEE 802.1cb-2017: IEEE standard for local and metropolitan area networks—frame replication and elimination for reliability, <https://standards.ieee.org/ieee/802.1CB/5703/> (September 2017).
- [9] M. Felser, Real-time ethernet-industry prospective, *Proceedings of the IEEE* 93 (6) (2005) 1118–1129.
- [10] S. Vitturi, C. Zunino, T. Sauter, Industrial communication systems and their future challenges: Next-generation ethernet, iiot, and 5g, *Proceedings of the IEEE* 107 (6) (2019) 944–961.

- [11] W. EtherCAT, Ethercat—the ethernet fieldbus (2012).
- [12] G. Prytz, A performance analysis of ethercat and profinet irt, in: 2008 IEEE International Conference on Emerging Technologies and Factory Automation, IEEE, 2008, pp. 408–415.
- [13] F. Hu, F. Qu, J. Li, H. Zhao, Real-time research based on arm-based robot ethercat master system, in: 2022 International Symposium on Control Engineering and Robotics (ISCER), IEEE, 2022, pp. 12–19.
- [14] P. R. Grams, Ethernet for aerospace applications-ethernet heads for the skies, Tech. rep. (2015).
- [15] K. Zambouri, M. Noor-A-Rahim, J. John, C. J. Sreenan, H. Vincent Poor, D. Pesch, A comprehensive survey of wireless time-sensitive networking (tsn): Architecture, technologies, applications, and open issues, IEEE Communications Surveys and Tutorials 27 (4) (2025) 2129–2155. doi:10.1109/comst.2024.3486618.
URL <http://dx.doi.org/10.1109/COMST.2024.3486618>
- [16] Time-sensitive networking (tsn) task group, <https://1.ieee802.org/tsn/>, accessed 2026-05-23.
- [17] A. Nasrallah, V. Balasubramanian, A. Thyagaturu, M. Reisslein, H. El-Bakoury, Reconfiguration algorithms for high precision communications in time sensitive networks: Time-aware shaper configuration with ieee 802.1qcc (extended version) (2019). arXiv:1906.11596.
URL <https://arxiv.org/abs/1906.11596>
- [18] S. S. Craciunas, R. S. Oliver, M. Chmélík, W. Steiner, Scheduling real-time communication in ieee 802.1 qbv time sensitive networks, in: Proceedings of the 24th International Conference on Real-Time Networks and Systems, 2016, pp. 183–192.
- [19] A. C. Santos, R. M. Silva, B. Schneider, M. Wilhelm, I. E. Fonseca, V. Nigam, A novel open-source framework for performing TSN schedules, Simulation Modelling Practice and Theory 144 (2025) 103147.
- [20] C. Xue, T. Zhang, Y. Zhou, M. Nixon, A. Loveless, S. Han, A survey and experimental study of real-time scheduling methods for 802.1 qbv TSN networks, ACM Computing Surveys 58 (2) (2025) 1–37.

- [21] Iec 62591:2016: Industrial networks - wireless communication network and communication profiles - wirelessharttm (2016).
- [22] I. S. Committee, et al., Isa100. 11a," wireless systems for industrial automation: Process control and related applications, Tech. rep., Technical Report, Research Triangle Park, North Carolina (2009).
- [23] J. T. Moy, OSPF: anatomy of an Internet routing protocol, Addison-Wesley Professional, 1998.
- [24] D. Allan, P. Ashwood-Smith, N. Bragg, J. Farkas, D. Fedyk, M. Ouellete, M. Seaman, P. Unbehagen, Shortest path bridging: Efficient control of larger ethernet networks, IEEE Communications Magazine 48 (10) (2010) 128–135.
- [25] D. Katz, D. Ward, Rfc 5880: Bidirectional forwarding detection (BFD), Tech. rep., Internet Engineering Task Force (2010).
- [26] Cisco Systems, Inc., IP routing: BFD configuration guide, Cisco IOS XE 17 (NCS 4200 series): Bidirectional forwarding detection, <https://www.cisco.com/c/en/us/td/docs/routers/ncs4200/configuration/guide/IProuting/17-1-1/b-irb-17-1-1-ncs4200/m-bfd-bidirectional-forwarding-ncs4200.html>.
- [27] Juniper Networks, Inc., Understanding how BFD detects network failures, <https://www.juniper.net/documentation/us/en/software/junos/high-availability/topics/topic-map/bfd.html>.
- [28] S. Bryant, C. Filsfils, S. Previdi, M. Shand, N. So, Rfc 7490: Remote loop-free alternate (LFA) fast reroute (FRR), Tech. rep., Internet Engineering Task Force (2015).
- [29] N. Finn, P. Thubert, B. Varga, J. Farkas, Deterministic networking architecture, Tech. rep. (2019).
- [30] C. Hopps, Analysis of an equal-cost multi-path algorithm, Tech. rep. (2000).
- [31] M. Alizadeh, T. Edsall, S. Dharmapurikar, R. Vaidyanathan, K. Chu, A. Fingerhut, V. T. Lam, F. Matus, R. Pan, N. Yadav, et al., Conga: Distributed congestion-aware load balancing for datacenters, in: Proceedings of the 2014 ACM conference on SIGCOMM, 2014, pp. 503–514.

- [32] N. Katta, M. Hira, C. Kim, A. Sivaraman, J. Rexford, Hula: Scalable load balancing using programmable data planes, in: Proceedings of the Symposium on SDN Research, 2016, pp. 1–12.
- [33] L. Tassiulas, A. Ephremides, Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks (1990) 2130–2132.
- [34] M. J. Neely, E. Modiano, C. E. Rohrs, Dynamic power allocation and routing for time varying wireless networks 1 (2003) 745–755.
- [35] M. Neely, Stochastic network optimization with application to communication and queueing systems, Morgan & Claypool Publishers, 2010.
- [36] S. Moeller, A. Sridharan, B. Krishnamachari, O. Gnawali, Routing without routes: The backpressure collection protocol, in: Proceedings of the 9th ACM/IEEE International Conference on Information Processing in Sensor Networks, 2010, pp. 279–290.
- [37] A. Warriier, S. Janakiraman, S. Ha, I. Rhee, Diffq: Practical differential backlog congestion control for wireless networks, in: IEEE INFOCOM 2009, IEEE, 2009, pp. 262–270.
- [38] Z. Zhao, G. Verma, A. Swami, S. Segarra, Enhanced backpressure routing using wireless link features, arXiv preprint arXiv:2310.04364 (2023).
- [39] T. Winter, P. Thubert, A. Brandt, J. Hui, R. Kelsey, P. Levis, K. Pister, R. Struik, J.-P. Vasseur, R. Alexander, Rpl: Ipv6 routing protocol for low-power and lossy networks, Tech. rep. (2012).
- [40] M. Goyal, E. Baccelli, M. Philipp, A. Brandt, J. Martocci, Reactive discovery of point-to-point routes in low-power and lossy networks, Tech. rep. (2013).
- [41] S. Bowe, simbit: A p2p network simulator, GitHub repository. Available: <https://github.com/ebfull/simbit>, commit ac861de (2016).

Appendix

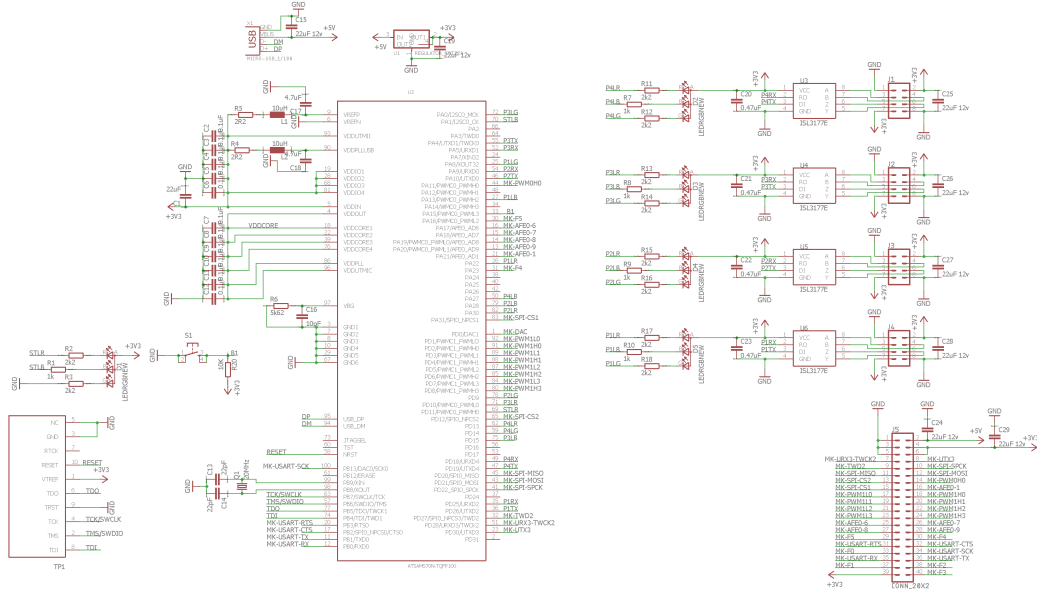


Figure A.1: ATSAM570-based TinyNet router schematic. Each UART port connects to an ISL3177E differential line driver for RS-485-compatible signaling. Port-select logic and buffer management run on the 300 MHz core.

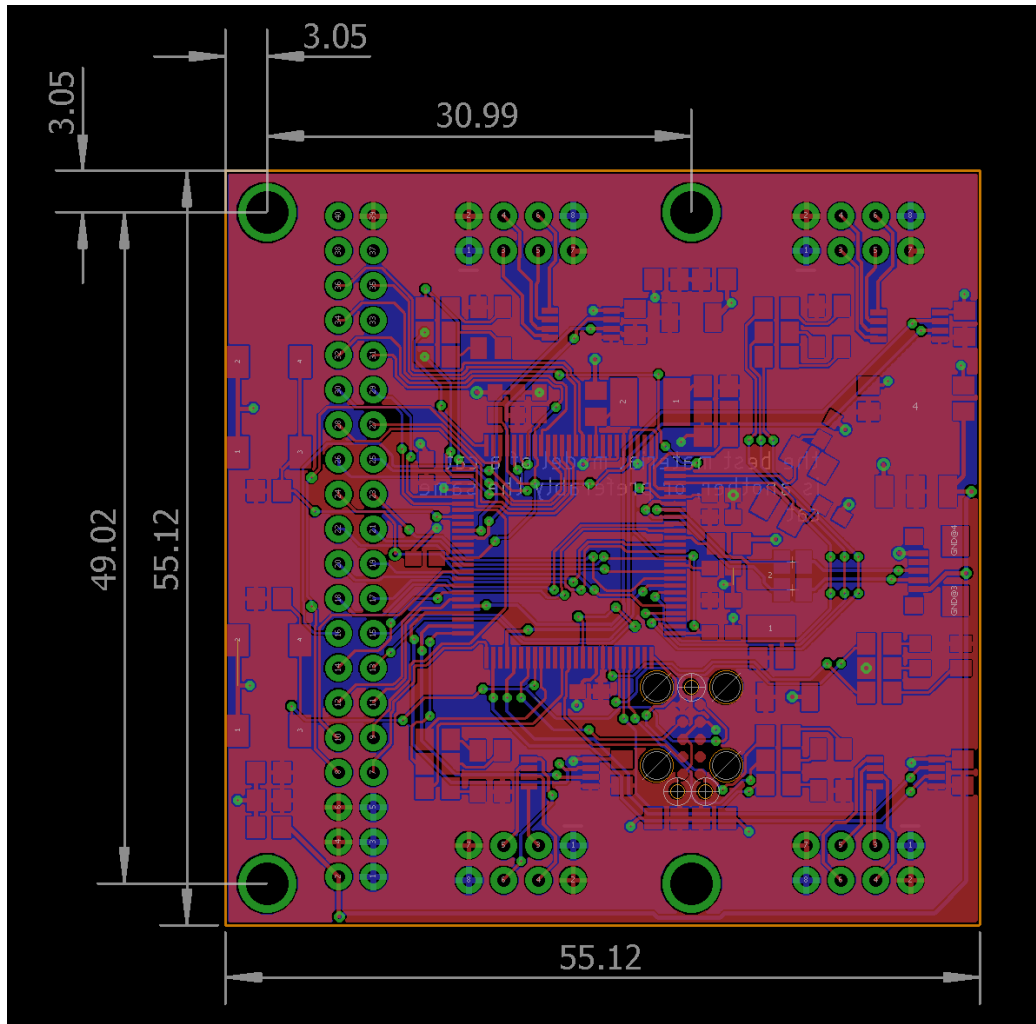


Figure A.2: TinyNet router PCB layout. The ATSAMS70 microcontroller occupies the center; ISL3177E drivers are arranged at the board periphery, one per UART port.